

AI Model with Road Network Planning

AI Model with Road Network Planning

1. Course Content
2. Preparation Work
 - 2.1 Start `kilted_agent` Agent
 - 2.2 Prerequisite Knowledge
 - 2.3 Configure Map Mapping File for Road Network Navigation Mode
3. Run Case Program
 - 3.1 Start Road Network Navigation
 - 3.2 Start `multi_brains` Agent
 - 3.3 Demonstration Case
4. Source Code Analysis

1. Course Content

- This course is a comprehensive advanced case study that combines AI large model functionality on the basis of road network planning navigation to achieve AI-controlled navigation within the road network

2. Preparation Work

2.1 Start `kilted_agent` Agent

- Use the following command to start the microROS agent and connect to the chassis

[!WARNING]

If you use the `start_agent` command to start the microROS agent, you need to close the previous agent first. Note that this is different from the previous microROS agent startup command

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

- Start ros2kilted container (if already started, no need to repeat)

```
bringup_ros2kilted
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 102x39
jetson@yahboom:~$ bringup_ros2kilted
access control disabled, clients can connect from any host
jetson@yahboom:~$
: Container ros2_kilted-ros2_kilted-1 Starting
1.4s
```

- Start Dify, all functions related to AI large models need to start Dify (if already started, no need to repeat)

```
bringup_dify
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 102x39
jetson@yahboom:~$ bringup_dify
[+] up 14/14
✓ Network docker_default Created 0.1s
✓ Network docker_ssrf_proxy_network Created 0.1s
✓ Container docker-sandbox-1 Created 0.1s
✓ Container docker-weaviate-1 Created 0.1s
✓ Container docker-db_postgres-1 Healthy 2.7s
✓ Container docker-ssrf_proxy-1 Created 0.2s
✓ Container docker-init_permissions-1 Exited 2.2s
✓ Container docker-web-1 Created 0.1s
✓ Container docker-redis-1 Created 0.1s
✓ Container docker-plugin_daemon-1 Created 0.0s
✓ Container docker-worker_beat-1 Created 0.1s
✓ Container docker-api-1 Created 0.1s
✓ Container docker-worker-1 Created 0.1s
✓ Container docker-nginx-1 Created 0.0s
jetson@yahboom:~$
```

[!WARNING]

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a container terminal,

```
exec_m3
```

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning navigation chapter needs to use cyclondds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK & PI5 mainboard

```
131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
145
```

- ORIN mainboard

```
133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
137
```

```
:wq #Save and exit
source ~/.bashrc #Update environment
```

2.2 Prerequisite Knowledge

For a better experience, please first learn and master the following content in the product tutorial:

- AI Large Model: 4. AI Large Model Development, [5.AI Large Model-Voice Version—03-Visual Understanding+SLAM Navigation]
- Road Network Planning Navigation: 7. Road Network Planning Navigation

2.3 Configure Map Mapping File for Road Network Navigation Mode

- For methods to obtain vehicle target points on the map, please refer to [05.AI Model-Voice Version]—[3.Multimodal visual understanding+SLAM navigation—2.3 Configure Map Mapping File]. The method is completely the same, with the difference being the fields filled in the `map_mapping.yaml1` map mapping file
- Edit the map mapping file (any editing method can be used)

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/multi_brains/map_mapping.yaml1
```

- Road network points need to be filled into the points under the `road_net_map_areas` field. Add or delete according to your needs

```
road_net_map_areas: #Road network navigation
C:
  name: 'School'
  position:
    x: 2.625
    y: -1.416
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.990
    w: 0.139

D:
  name: 'Safe Area'
  position:
    x: 1.583
    y: -0.455
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -1.000
    w: -0.007

E:
  name: 'xxx'
  position:
    x: 0.881
    y: -1.369
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.004
    w: 1.000
```

- Additionally, you need to update the new map mapping information in Dify and publish save settings. For configuration methods, refer to [05.AI Model-Voice Version]—[3.Multimodal visual understanding+SLAM navigation—2.4 Configure Map Mapping Variables in Dify]

Edit Conversation Variable

Name
map_mapping

Type
array[string]

Default Value [Edit in JSON](#)

School: A

Library: B

+ Add Value

Description

Map mapping stores locations that need to be known in advance by AI

Cancel Save

3. Run Case Program

[!IMPORTANT]

- Need to build the track map's grid map: map, pose map: pose_map, and road network description file: roadnet_file in advance according to [08.Autonomous Driving expand(Need purchase track map)]—[13.Road Network Planning with Track Map]

3.1 Start Road Network Navigation

- Open a terminal on the host machine to start the vehicle's radar fusion node and odometer fusion
- (PI5 users start this command in the `m3` container)

```
ros2 launch m3_bringup car_base.launch.py
```

- Start `ros2_kilted` container

```
bringup_ros2kilted
```

- Open ros2kilted container terminal

```
exec_ros2kilted
```

- (RDK & PI5 mainboard) Visualization is started on the accompanying virtual machine
`Rosmaster-RoadNet-Ubuntu24.04`,

```
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/nav2_view.rviz
```

- Start road network planning navigation node in container terminal

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

[!NOTE]

Parameter Description

map Grid map file name to be loaded, default value is `map.yaml`

pose_map Pose map file to be loaded. When navigation global positioning uses slam_toolbox pure positioning, the pose map must be passed in, default value is `map`

roadnet_file Road network description file, default value is `map.geojson`

3.2 Start multi_brains Agent

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a container terminal,

```
exec_m3
```

Open a terminal on the vehicle and enter the command to start:

```
ros2 launch m3_bringup llm_agent_control.launch.py enable_route_nav:=True
```

```

[model service-12] ALSA lib pcm.c:2664:(snd_pcm_open nouupdate) Unknown PCM cards.pcm.modem
[model service-12] ALSA lib pcm.c:2664:(snd_pcm_open nouupdate) Unknown PCM cards.pcm.modem
[model service-12] ALSA lib pcm.c:2664:(snd_pcm_open nouupdate) Unknown PCM cards.pcm.phoneline
[model service-12] ALSA lib pcm.c:2664:(snd_pcm_open nouupdate) Unknown PCM cards.pcm.phoneline
[model service-12] Cannot connect to server socket err = No such file or directory
[INFO] [model service-12]: process started with pid [116868]
[INFO] [action service usb-13]: process started with pid [116870]
[action service usb-13] [INFO] [1766401793.044243388] [action service usb]: enable route nav: True
[action service usb-13] [INFO] [1766401793.152994034] [action service usb]: ROS Action Service Initialization completed
[model service-12] [INFO] [1766401802.719665243] [model service]: Diffy LLM connection successful
[model service-12] [WARN] [1766401802.720780204] [rcl.logging rosout]: Publisher already registered for provided node name. If this is due to multiple nodes with the same name then all logs for that logger name will go out over the existing publisher. As soon as any node with that name is destructed it will unregister the publisher, preventing any further logs for that name from being published on the rosout topic.
[model service-12] ALSA lib pcm.c:2664:(snd_pcm_open nouupdate) Unknown PCM cards.pcm.rear
[model service-12] ALSA lib pcm.c:2664:(snd_pcm_open nouupdate) Unknown PCM cards.pcm.center_lfe
[model service-12] ALSA lib pcm_oss.c:397:(snd_pcm_oss_open) Cannot open device /dev/dsp
[model service-12] ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
[model service-12] ALSA lib pcm_usb_stream.c:482:(snd_pcm_usb_stream_open) Invalid card 'card'
[model service-12] ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
[model service-12] ALSA lib pcm_usb_stream.c:482:(snd_pcm_usb_stream_open) Invalid card 'card'
[model service-12] ALSA lib pcm_dmix.c:1005:(snd_pcm_dmix_open) The dmix plugin supports only playback stream
[model service-12] Cannot connect to server socket err = No such file or directory
[model service-12] Cannot connect to server request channel
[model service-12] jack server is not running or cannot be started
[model service-12] JackShmReadWritePtr::~JackShmReadWritePtr - Init not done for -1, skipping unlock
[model service-12] JackShmReadWritePtr::~JackShmReadWritePtr - Init not done for -1, skipping unlock
[model service-12] [INFO] [1766401803.247590096] [ASR_Detect]: paraformer-realtime-v2 ASR Engine initialized successfully
[model service-12] [INFO] [1766401803.262672610] [model service]: Tongyi TTS Engine initialized successfully
[model service-12] [INFO] [1766401803.263389574] [model service]: ROS LLM Service Initialization completed

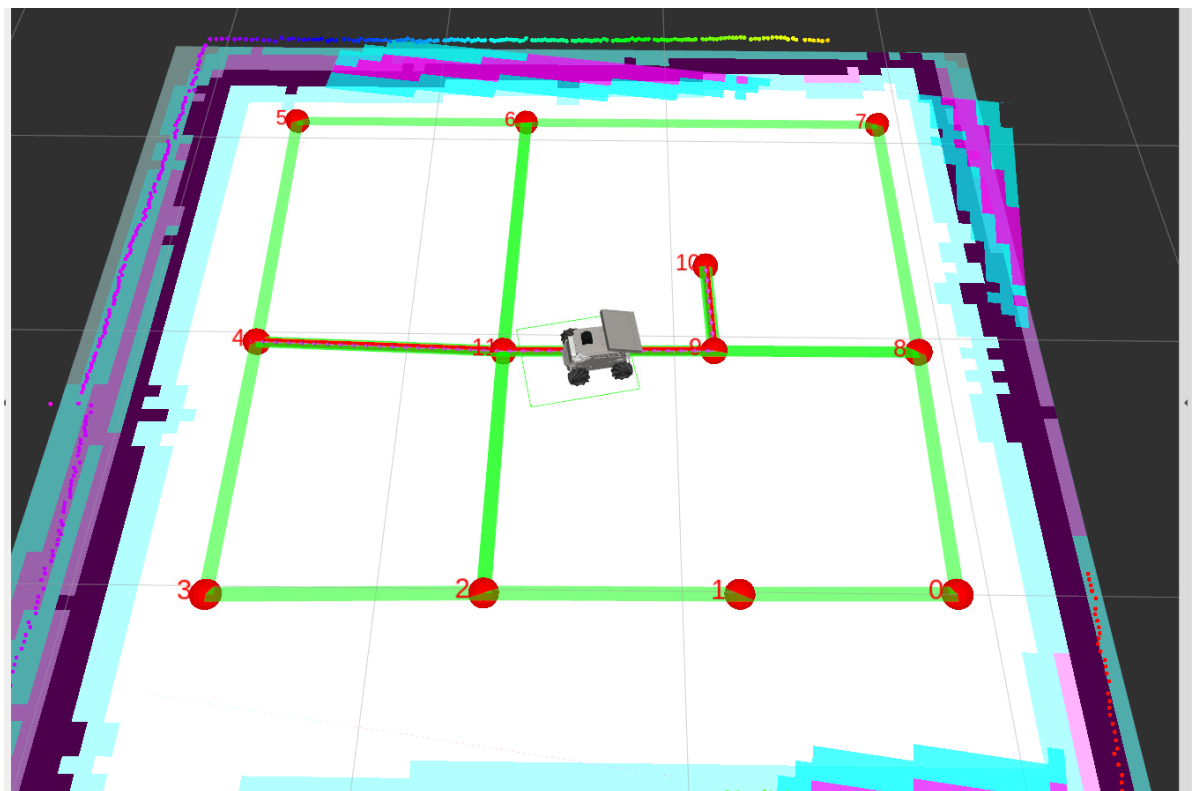
```

3.3 Demonstration Case

- Navigate to the school and observe the surrounding environment

After waking up the vehicle with "Hello, yahboom", when the vehicle responds and the recording prompt sound "dang—" appears, give the command to the vehicle

- The vehicle will then follow the road network planned route to the target point "School" and observe the environment. The route will only travel within the road network



4. Source Code Analysis

Source code path:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/action_service.py
```

- In the action_service node initialization, the load_target_points method will be called to load the map_mapping.yaml map mapping file, and the content of the road_net_map_areas field in the configuration file will be stored in the self.road_net_dict variable.

```
def load_target_points(self):
    """
    Load map mapping file
    """
    self.navpose_dict = {}
    self.road_net_dict = {}
    with open(self.map_mapping_file, "r") as file:
        full_target_points = yaml.safe_load(file)
        common_target_points = full_target_points.get("common_map_areas",
    {})

        road_net_target_points =
full_target_points.get("road_net_map_areas", {})

-----

    for name, data in road_net_target_points.items():#Load road network
navigation points
        pose = PoseStamped()
        pose.header.frame_id = "map"
        pose.pose.position.x = data["position"]["x"]
        pose.pose.position.y = data["position"]["y"]
        pose.pose.position.z = data["position"]["z"]
        pose.pose.orientation.x = data["orientation"]["x"]
        pose.pose.orientation.y = data["orientation"]["y"]
        pose.pose.orientation.z = data["orientation"]["z"]
        pose.pose.orientation.w = data["orientation"]["w"]
        self.road_net_dict[name] = pose
```

- When the AI large model calls the navigation method, it will judge based on the startup parameter enable_route_nav. If road network navigation mode is started, it will use the __road_net_navigation method for navigation
- __road_net_navigation will construct target pose information and send road network navigation target requests to the route_bridge node through the road_net_nav_pub topic publisher for road network navigation

```
def navigation(self, point_name):
    """
    Get target point coordinates from navpose_dict dictionary and navigate to
    the target point
    """
    if self.enable_route_nav:
        # Use road network navigation
        if self.__road_net_navigation(point_name):
            return True
        else:
            return None
    else:
        # Use normal navigation
        if self.__normal_navigation(point_name):
            return True
```



```

        else:
            return None

def __road_net_navigation(self, point_name):
    '''Road network navigation function'''
    self.road_net_nav_future = Future()
    goal_msg=PoseStamped()
    #Construct endpoint
    point_name = point_name.strip("\'\"")
    goal_msg = self.road_net_dict.get(point_name)
    self.road_net_nav_pub.publish(goal_msg)

    while not self.road_net_nav_future.done():
        if self.interrupt_event.is_set():
            self.cancel_nav_pub.publish(Bool(data=True))
            self.get_logger().info(Fore.GREEN+"Road net navigation
None"+Fore.RESET)
            time.sleep(0.1)
        result = self.road_net_nav_future.result()
        self.__disable_alarm()
        if result.data == "road_net_nav_succeeded":
            self.get_logger().info(Fore.GREEN+"Road net navigation
succeeded"+Fore.RESET)
            return True
        else:
            return False

```